Proyecto 3: Objectilang

Integrantes:
Carolina Ceballos Gómez - 202510267
Juan Felipe León Barrero - 202510397

1. Correcciones modelo de la entrega 2:

Para esta entrega trabajamos sobre nuestro proyecto de la entrega anterior, con las correcciones de la retroalimentación de la monitora, en especial aquellas que causaban que los snippets no parsearan..

Arreglamos el new, que siempre pedía paréntesis, con una regla aparte para el new sin paréntesis. No lo hicimos opcional dentro de la misma regla porque quedaba ambiguo con una llamada. También cambiamos la asignación de métodos, el primer nombre de la ruta puede ser una clase (Cat.talk) y el bloque pasó a ser opcional, porque en el snippet 3 esa asignación no lleva cuerpo. El if ahora también puede usarse como valor (val max = if a > b then a else b), con el else obligatorio para evitar ambigüedades. Además, agregamos como sentencias el new solo (new DogFood) y la variable suelta.

También corregimos que negate se leyera como neg ate, que la interpolación no aceptara espacios (${ x }) y también el resaltado, que estaba con mayúscula y el Rascal solo reconoce las de minúscula.

Después de eso hicimos otros ajustes por problemas que nos fueron apareciendo.
En especial, de renombre para que no hubiera nombres duplicados, renombrámos tres nodos que chocaban: el operador % pasó de módulo a resto, porque módulo ya era el nodo del programa completo, el cuerpo de un método con bloque pasó de cuerpo a cuerpoBloque, y el or pasó de operation a oLógico.

Algunas cosas siguen igual porque el enunciado no las especifica y las tomamos como decisiones de diseño: el menos unario, el main al final del programa y el -> solo como parte de los tipos de función. Además, el AST guarda los strings y los caracteres sin las comillas, porque el imploder se las quita al construir el árbol.

2. Justificación de Diseño Proyecto 3

Seguimos la estructura del tutorial: la gramática (Syntax.rsc), el AST con un constructor por cada regla (AST.rsc), el parser y el imploder para pasar del texto al AST, un generador de código sobre el AST y TypePal para el análisis de tipos y nombres.

Agregamos el tipo Char, con literales como 'a' en la gramática y en el AST (caracter). Las anotaciones de tipo ya las teníamos (val x: Int) y los tipos genéricos que da el usuario se escriben como object Caja[T] o List[Int].

Para ejecutar archivos .ol escribimos Generator.rsc, que sigue la misma idea de los generadores de la sección 6 del tutorial, recorre el AST y produce un resultado.Pero, en el tutorial el resultado es un archivo .txt, mientras que nuestro generador imprime el resultado directamente en la consola (así lo pide el enunciado), se usa con generar(archivo). Soporta variables y asignaciones, aritmética, comparaciones, and, or y neg, if como sentencia y como valor, etc. También ejecuta objetos: new con y sin paréntesis y con tipos genéricos, parámetros y atributos, métodos, herencia con extends y override, métodos asignados con Clase.metodo = method(...), objetos anónimos (new Animal("x") { ... }), toString y getClass.getSimpleName.

Instalamos TypePal en el proyecto siguiendo la sección 9 del tutorial y escribimos Checker.rsc. Igual que en el tutorial, extiende el módulo de TypePal, definimos tipos propios y roles, usamos patrones de sintaxis concreta para registrar definiciones y usos, y tiene una función que arma el modelo de tipos (objectilangTModel). Los tipos son Int o Integer , Boolean, String, Char, listas, clases y genéricos, y la herencia se trata como subtipo: val p: Animal = new Perro(...). Verificamos que el tipo de cada valor coincida con su anotación, que las condiciones de los if sean Boolean, que los operadores reciban los tipos correctos y que las asignaciones respeten el tipo de la variable.

También definimos una regla nueva para asegurarnos de que lo que se usa en la definición de un objeto exista, por ejemplo, la clase padre de un extends, los tipos de los parámetros y atributos, la clase en un new y la clase en Clase.metodo tienen que estar definidos, y toda variable usada tiene que estar declarada. Para probar esta regla, escribimos errores_tipos.ol, programa con errores que debe marcar, o ejemplo1.ol, ejemplo2.ol y ejemplo3.ol, que no deben dar errores.

Inicialmente los atributos heredados no se veían desde los métodos de una subclase, pero usamos los caminos de TypePal, ahora al definir una clase con extends, la conectamos (con addPathToDef) con su clase padre y con todos sus ancestros, y así una subclase ve los atributos, parámetros y métodos heredados.

Tuvimos que actualizar el Plugin.rsc para que el editor reconociera archivos .ol (y .objl y .obj, que eran algunas de las pruebas que teníamos del proyecto 2).

Aclaración importante:
Para esta entrega, utilizamos Rascal 0.43.0 y TypePal 0.17.3, que son versiones mucho más recientes que las que aparecen en el tutorial, ya que cuando utilizábamos las dadas en el tutorial, nos aparecían varios errores. Por esto también, ya no declaramos en el RASCAL.MF, sino que lo manejamos como una dependencia de Maven directamente en el pom.xml. Además, también cambiaron algunos detalles de sintaxis del código (en Plugin). En el tutorial se usan parser y summarizer, pero en nuestra versión se utilizan parsing y analysis.

3. Aclaración sobre las pruebas realizadas

Para asegurarnos de que el Checker y el Generador estuvieran funcionando, decidimos hacer algunas pruebas. Para esto, nos apoyamos un poco de la IA para que nos generara archivos .ol con bastantes casos distintos, algunos que debían fallar a propósito y otros que debían dar un resultado correcto en consola. Queremos aclarar que el uso de la IA fue totalmente hecho a conciencia y como una herramienta de apoyo para generar los archivos de prueba. El prompt que le dimos fue el enunciado del Proyecto 1 (donde están los snippets de la gramática) y una explicación de lo que habíamos hecho en el Proyecto 3, y le pedimos que, con base en eso, nos generara archivos de prueba para diferentes casos. Estos casos se encuentran en la carpeta "instance" por si los desean revisar u probar.

4. Uso de Soluciones Previas del Proyecto 2

Dado que nuestro grupo sigue estando conformado por exactamente las mismas personas que en el Proyecto 2, la gramática y el proyecto Rascal que utilizamos en la solución presentada son de nuestra autoría original y no provienen de ningún otro grupo. Sin embargo, al recibir bastantes correcciones del proyecto 2, si realizamos muchos cambios (explicados en la sección 1 de este documento).